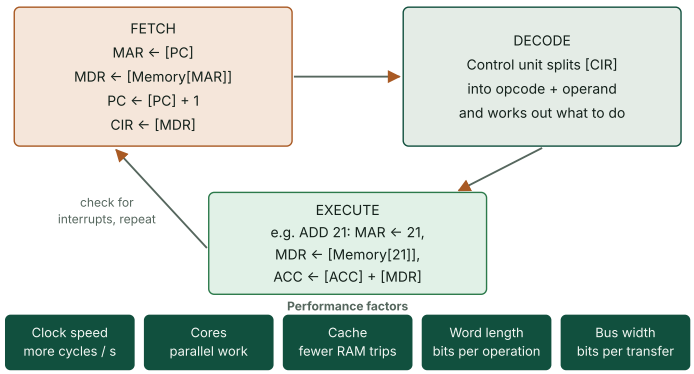
The fetch–decode–execute cycle, pipelining and processor performance

FD09 FACT FILE

Specification: students should be able to

- Describe the fetch–execute cycle, including the fetch, decode and execute stages and the roles of the registers and buses
- Describe how pipelining overlaps the stages of the cycle, and calculate the time units needed with and without pipelining
- Evaluate the factors that influence the speed of processing: processor type (word size, number of cores) and clock speed
- Explain how cache size, bus width and the amount of RAM also affect performance
- Explain why clock speed alone is not a good measure of performance, and describe benchmark measures such as MIPS and FLOPS
- Interpret a computer specification, such as '64-bit 3.2 GHz quad-core'

Key words: fetch–decode–execute cycle, fetch, decode, execute, instruction set, opcode, operand, pipelining, clock speed, hertz (Hz), multi-core processor, word size, 32-bit / 64-bit processor, cache, bus width, benchmark, MIPS, FLOPS



The fetch-decode-execute cycle shown as a loop with the register transfers at each stage, and the factors that affect processor performance

The cycle in overview

Every program, from a C# console app to a web browser, is run by the processor repeating the same three-stage routine for each instruction: fetch, decode and execute. Before the cycle can begin, the operating system copies the program's instructions and data from secondary storage into main memory, because the processor can only fetch directly from RAM. In the fetch stage the next instruction is copied from memory into the processor. In the decode stage the control unit works out what the instruction means. In the execute stage the instruction is carried out, which might involve a calculation by the ALU, reading or writing memory, or changing which instruction comes next. The cycle then repeats, continuously, from power-on until shutdown, and a modern processor completes billions of these cycles every second. The routine is often called the fetch–execute cycle, but it has three distinct stages. The speed at which a processor gets through the cycle depends on its clock speed, how many cores it has, its word size, its cache and how quickly memory can supply it, while pipelining lets the stages of different instructions overlap so more work is completed in the same time.

Fetch in detail

The fetch stage uses the special-purpose registers and all three buses. The program counter always holds the address of the next instruction to be fetched; when a program starts it holds the address of the first instruction. That address is copied from the program counter into the memory address register. The program counter is then incremented so that it already points to the instruction that follows, ready for the next cycle. The memory address register places the address on the address bus, and the control unit sends a memory-read signal along the control bus. Main memory responds by placing the contents of that location on the data bus, and they arrive in the memory data register. Finally, the instruction is copied from the memory data register into the current instruction register, where it will be decoded. Textbooks differ over exactly when the program counter is incremented, but it must happen during the fetch and after its contents have been copied to the memory address register. In an exam answer, keep the steps in order and name each register and bus in full, because a sequence that misses the increment or muddles the registers will lose marks.

Decode and execute

In the decode stage the control unit interprets the instruction held in the current instruction register. A machine-code instruction has two parts: the opcode, which identifies the operation, such as load, add, store or jump, and the operand, which is either a value to be used or the address where the value is found. Each processor family has its own instruction set, the complete list of opcodes it recognises, which is why a program compiled for one family cannot run on a different one. Having decoded the instruction, the control unit sets up the registers and circuits needed. In the execute stage it sends the control signals that carry the instruction out. If the operand is an address, that address is placed in the memory address register so that data can be read into, or written out of, the memory data register. Arithmetic and logical operations are performed by the ALU, with the result held in the accumulator. A jump or branch instruction works differently: it replaces the contents of the program counter, so the next fetch comes from a new part of the program. This is how selection and loops in high-level code are carried out at machine level.

Pipelining

In a simple processor each instruction is fully fetched, decoded and executed before the next fetch starts, so with three stages of one time unit each, n instructions take 3n time units. Pipelining overlaps the stages of different instructions. As soon as the first instruction moves on to be decoded, the second is fetched; when the first is executed, the second is decoded and the third is fetched. Once the pipeline is full, one instruction completes in every time unit, so n instructions need only n + 2 units. Four instructions therefore take six units instead of twelve, and three take five instead of nine. Each individual instruction still takes three units, but throughput, the number of instructions completed per second, rises without any increase in clock speed. The method works best when instructions follow each other in order. A branch or jump means the instructions already fetched and decoded behind it may be the wrong ones, so the pipeline has to be emptied and refilled, wasting some time units. Modern processors use pipelines with many more than three stages, along with techniques to predict which way a branch will go, but the principle is the same.

Clock speed and processor type

The clock produces a regular series of pulses, and each step of the cycle takes one or more of them. Clock speed is the number of cycles per second, measured in hertz, so a 3.2 GHz processor completes 3.2×10^9 cycles every second. Raising the clock speed lets more instructions be processed per second, but it also increases power consumption and heat; a chip that gets too hot can be damaged, so faster processors need better cooling and drain laptop batteries sooner. Processor type matters as much. Word size is the number of bits the processor treats as a single unit, matching the size of its general registers. A 64-bit processor handles larger values in one operation and, unlike 32-bit addressing, which is limited to 2^{32} bytes or 4 GB, can make use of far more memory. A 64-bit processor will run 32-bit software, but programs built for 64-bit cannot run on a 32-bit processor, and a 32-bit operating system cannot use a 64-bit processor fully. A multi-core processor contains two or more independent cores on one chip, each able to run its own stream of instructions, so several threads or programs run at once, provided the software has been written to divide its work.

Other factors and benchmarks

Several other features affect how quickly a computer processes data. Cache memory holds frequently used instructions and data beside the processor; a larger cache means fewer slow trips to RAM, although cache is costly and the benefit levels off. A wider data bus moves more bits in each transfer. More RAM means the operating system has to swap data out to slow secondary storage less often. Because so many factors interact, comparing processors by clock speed alone is misleading: two chips running at the same speed can differ greatly in how many instructions they complete per cycle, how deep their pipeline is, how many cores they have and how much cache they carry. Upgrading one component can also simply move the bottleneck elsewhere, for example a faster processor left waiting for a slow hard disk. A fairer comparison uses a benchmark, a standard set of tasks run on each machine with the results timed. Processor benchmarks are often reported in MIPS (millions of instructions executed each second) or in FLOPS (the number of floating-point calculations completed each second), which matters for science, graphics and machine learning. These figures describe the processor only and ignore storage and network speed.

Key terms from the mark schemes

Fetch–decode–execute cycle
The continuous process by which the CPU fetches an instruction from memory, decodes it and carries it out.
Credited: Fetch–execute cycle / instruction cycle

Operand
The part of a machine-code instruction that holds the data to be used or the address where the data is stored.

Pipelining
Overlapping the fetch, decode and execute stages of successive instructions so that several instructions are being processed at once.

Core
An independent processing unit within a CPU, able to fetch, decode and execute its own instructions.

Word size
The number of bits a processor can handle as a single unit in one operation, usually equal to the size of its registers.
Credited: Word length

Benchmark
A standard set of tasks run on different computers so that their performance can be measured and compared fairly.

Opcode
The part of a machine-code instruction that specifies the operation to be performed.
Credited: Operation code

Instruction set
The complete set of machine-code instructions that a particular processor is designed to recognise and execute.

Clock speed
The number of clock cycles the processor carries out per second, measured in hertz (e.g. 3.5 GHz).
Credited: Clock rate

Multi-core processor
A single processor chip containing two or more independent cores that can execute instructions at the same time.
Credited: Dual-core / quad-core

Cache
Small, very fast memory close to the CPU that stores frequently used instructions and data to reduce slower accesses to RAM.

FLOPS
Floating-point operations per second: a measure of a processor's speed at arithmetic on real numbers.
Credited: MIPS (millions of instructions per second) for general instructions

C C# example

Simulating the cycle in C#

```
int[] memory = { 104, 205, 306, 0, 7, 5, 0 };
int pc = 0, acc = 0;
while (true)
{
    int mar = pc;                 // fetch: MAR <- PC
    int mdr = memory[mar];        // MDR <- memory[MAR]
    pc++;                         // PC <- PC + 1
    int cir = mdr;                // CIR <- MDR
    int opcode = cir / 100, operand = cir % 100;   // decode
    if (opcode == 0) break;                        // HLT
    if (opcode == 1) acc = memory[operand];        // LDA
    else if (opcode == 2) acc += memory[operand];  // ADD
    else if (opcode == 3) memory[operand] = acc;   // STA
    Console.WriteLine($"PC={pc} CIR={cir} ACC={acc}");
}
Console.WriteLine($"Address 6 holds {memory[6]}");
```

Exam focus

Fetch–execute questions are usually 'describe' or 'explain' items worth three to six marks, with one mark for each correct, correctly ordered step. Name registers in full and say what moves where, for example 'the contents of the program counter are copied to the memory address register', rather than vague phrases like 'the PC goes to memory'. Remember to include the increment of the program counter, the read signal on the control bus and the copy from the memory data register to the current instruction register. For decode, mention opcode and operand; for execute, mention the ALU and the accumulator. Pipelining questions may ask for a diagram: draw time units down the side and instructions across, with stages staggered by one unit, and state the total. Performance questions reward a reason linked to an effect: 'more cores allow several threads to be executed at the same time' rather than 'more cores is faster'. When asked why clock speed alone is a poor measure, explain that processor architecture, cores, cache and pipelining differ. Clock speeds use decimal prefixes (1 GHz = 10^9 Hz); if a question sets a different convention, follow it and show your working.

? Check yourself

1. State the three stages of the fetch–execute cycle. [1]
2. Describe the fetch stage of the fetch–decode–execute cycle, naming the registers and buses involved. [6]
3. Explain what happens during the decode stage, referring to the opcode and operand. [3]

Answers: 1. Fetch, decode, execute (all three for 1 mark) 2. The address in the program counter (PC) is copied into the memory address register (MAR) (1) / the PC is incremented to point to the next instruction (1) / the address held in the MAR is put onto the address bus (1) / a read signal is sent on the control bus (1) / the instruction at that address is sent along the data bus into the memory data register (MDR) (1) / the instruction is copied from the MDR into the current instruction register (CIR) (1) 3. The control unit interprets the instruction in the CIR (1) / it splits it into the opcode, which identifies the operation to perform (1) / and the operand, which is the data or the address of the data (1) / (accept: prepares registers for execution)